AI-BASED FACIAL AUTHENTICATION SYSTEM

with Liveness Detection and Deepfake Protection

React • FastAPI • ArcFace • VLM Hybrid Reasoning

PROJECT REPORT

Academic Year 2025–2026

Indian Institute of Information Technology Dharwad

Department of Computer Science and Engineering

PROJECT TEAM

S.No	Student Name	Role / Contribution
1	J Ganesh Kumar Reddy	Frontend Development (React)
2	B Varshith	Backend & AI Pipeline (FastAPI)
3	B Harsha Vardan	Liveness & Deepfake Detection
4	M Jagadeeswar	VLM Integration & Security

Team Details

- **Team Name:** Team Maya
- **Member 1:** J Ganesh Kumar Reddy (23BDS024)
- **Member 2:** B Varshith (23BDS011)
- **Member 3:** B Harsha Vardan (23BDS015)
- **Member 4:** M Jagadeeswar (23BDS033)

Abstract

Facial authentication is convenient but vulnerable to attacks such as printed photos, screen replays, recorded videos, and AI-generated faces. This project implements a comprehensive, full-stack facial authentication system that combines face recognition with liveness detection, anti-spoofing, and deepfake screening to address these critical vulnerabilities.

The system uses a React frontend to capture user face data through the browser camera and a FastAPI backend to process images and videos through a layered AI pipeline. Face detection is performed with OpenCV YuNet, face recognition is carried out using ArcFace embeddings generating 512-dimensional identity vectors, and authentication decisions are based on multiple independent signals including face similarity, liveness score, deepfake probability, and optional challenge-response verification.

Biometric templates are stored as encrypted 512-dimensional embeddings using AES-256-GCM, and all authentication attempts are recorded in SQLite for auditability. Successful authentication generates an RS256 JWT for downstream API access.

The latest version of the system additionally integrates a Vision Language Model (VLM) hybrid reasoning layer that compares stored registration reference frames with current authentication frames, returning both numerical confidence scores and natural-language explanations. The project demonstrates how practical biometric authentication can be hardened against photo attacks, video replay attacks, and AI-generated face attacks through a multi-layered, explainable pipeline.

Key Contribution: A six-gate decision engine requiring agreement from multiple independent AI signals — making single-point bypass attacks substantially more difficult than conventional face recognition systems.

1. Introduction

Facial authentication systems have become ubiquitous in modern computing, appearing in smartphones, laptops, access control systems, and financial services. Their appeal lies in the combination of user convenience and the uniqueness of human faces as biometric identifiers. However, face recognition alone is insufficient for secure authentication because it only answers whether two face images are visually similar — it does not prove that the face is live, captured from a real person present at the time of authentication, or free from synthetic manipulation.

Adversarial attacks against face authentication systems are well-documented and increasingly accessible. A printed photograph, a mobile screen replay, a pre-recorded video, a virtual camera feed injecting synthetic content, or an AI-generated deepfake can all fool systems that rely exclusively on face similarity scores. As generative AI models become more capable and widely available, the risk of synthetic face attacks increases further. The proliferation of deepfake tools has dramatically lowered the barrier for mounting successful spoofing attacks.

This project addresses these limitations by combining identity verification with multiple independent security checks organized into a coherent pipeline. The core insight is that robust authentication requires agreement from several security signals rather than depending on any single model. The layered approach makes the system more resistant to single-point failure and produces more explainable decisions through individual score outputs.

The system is implemented as a full-stack application with a React browser interface, a FastAPI backend, and a modular AI pipeline that runs face detection, face recognition, liveness scoring, deepfake risk estimation, optional challenge-response verification, and an optional Vision Language Model reasoning layer. Biometric data is protected through encryption and all authentication decisions are logged for audit.

1.1 Motivation and Context

Traditional face authentication systems typically depend on a single face similarity score, which is vulnerable to a range of well-known attack categories. The multi-layer approach presented in this project makes the system substantially more robust: an attacker must simultaneously fool the face similarity check, the liveness detector, the deepfake detector, and optionally the active challenge system — which dramatically raises the difficulty of a successful attack.

The system also provides academic value through its combination of digital signal processing concepts (FFT and rPPG), computer vision for detection and alignment, deep learning for embeddings and CNN features, and cybersecurity principles including spoofing resistance, encrypted biometric storage, JWT-based access control, and audit logging.

2. Problem Statement

The goal is to build a secure and explainable facial authentication system that can address the full spectrum of common face spoofing attacks. Existing face-based login systems typically depend on a single face similarity score, which is vulnerable to the following categories of attack:

- Printed photo attacks: An attacker presents a photograph of the registered user to the camera, either printed on paper or displayed on a screen.
- Screen replay attacks: A recorded video or displayed image of the user is played back to the camera in real time.
- Recorded video replay: A pre-recorded video of the user is submitted through a physical or virtual camera device.
- Virtual camera injection: The attacker uses software to replace the physical camera feed with synthetic or pre-recorded content, bypassing physical presence requirements.
- AI-generated faces and deepfakes: Generative AI creates hyper-realistic synthetic faces or face swaps that visually match the registered user, making identity spoofing trivially accessible.
- Similar-looking impostors: Individuals who are physically similar to the registered user attempt authentication, exploiting similarity-based recognition thresholds.

A secure facial authentication system must not only verify who the user appears to be but also verify the authenticity of the submitted biometric sample. This requires detecting spoofing attempts, synthetic content, and replay attacks in addition to performing identity matching. Furthermore, the system must store biometric templates securely so that a database breach does not expose raw facial data.

3. Objectives

3.1 Primary Objectives

1. Build a working face registration and authentication system accessible through a web browser.
2. Detect and align faces from browser-captured images and videos using OpenCV YuNet.
3. Extract robust 512-dimensional face embeddings using ArcFace (w600k_r50.onnx via ONNX Runtime).
4. Store biometric templates securely using AES-256-GCM encryption.
5. Verify identity using cosine similarity against a registered encrypted template.

6. Detect spoofing risk through passive and video-based liveness analysis.
7. Detect deepfake risk using frequency-domain, CNN-feature, texture, reflection, boundary, and temporal cues.
8. Return clear authentication decisions with individual scores, threat flags, and denial reasons.

3.2 Secondary Objectives

- Provide a clean web interface for registration and login with real-time score visualization.
- Support video-based authentication for stronger multi-frame temporal analysis.
- Include optional instruction challenge workflows for active liveness verification.
- Maintain a SQLite audit log of all authentication attempts with full score history.
- Provide training scripts for fine-tuning liveness and deepfake classification models.
- Extend the system with optional VLM hybrid reasoning for improved explainability and natural-language audit trails.
- Document the full architecture, methodology, setup, limitations, and future scope.

4. System Architecture

4.1 Architecture Overview

The project uses a layered full-stack architecture with clear separation of responsibilities across six distinct layers: presentation, API, pipeline, AI model, security, and data. This separation makes each layer independently understandable, testable, and replaceable without affecting the others.

The high-level data flow proceeds as follows: the user browser captures webcam frames or records a short video, which is transmitted to the React frontend. The frontend manages camera access, frame capture, video recording, and result display. Uploaded media is sent to the FastAPI backend via multipart form-data over HTTP. The backend routes requests to the AuthPipeline, which orchestrates all AI modules and applies the decision engine before returning results to the frontend.

4.2 Frontend Architecture

The frontend is implemented with React 18, Vite, Axios, and React Router. It provides four main user-facing pages and reusable camera components:

Route	Component	Backend Endpoint
/register	Register.jsx	/api/v1/register
/login	Login.jsx	/api/v1/authenticate/video
/vlm-register	VLMRegister.jsx	/api/v1/vlm/register

/vlm-login	VLMLogin.jsx	/api/v1/vlm/authenticate

4.3 Backend Architecture

The backend is implemented with FastAPI and SQLAlchemy. Route handlers decode uploaded media, validate user state, delegate to the authentication pipeline, manage encryption and JWT issuance, and persist audit records. Key backend files include:

- app/main.py: API entry point, startup hooks, and router registration.
- app/pipeline.py: AuthPipeline class orchestrating all model modules and the decision engine.
- app/config.py: Centralized thresholds, model paths, security settings, and pipeline configuration.
- app/crypto.py: AES-256-GCM helpers for embedding encryption and RS256 JWT helpers.
- app/db/models.py: SQLAlchemy schema for users, auth_logs, and challenge_logs.
- app/db/crud.py: Database access helpers for registration, retrieval, and audit logging.

4.4 AI Pipeline Layers

The AuthPipeline executes six sequential layers, each handling a distinct security concern:

Layer	Component	Function
Layer 0	Anti-Injection Guard	Detect virtual or injected camera sources via PRNU noise analysis and metadata inspection
Layer 1	Face Detection & Alignment	YuNet detects face and five landmarks, aligns to 112x112 canonical crop
Layer 2	Face Recognition	ArcFace extracts 512-D embeddings, cosine similarity match against stored template
Layer 3	Liveness Detection	CNN features, texture, color, optical flow, rPPG pulse estimation, micro-movement fusion
Layer 4	Deepfake Detection	FFT, EfficientNet features, boundary, eye reflection, skin uniformity, RGB correlation, temporal flicker
Layer 5	Instruction Verification	MediaPipe FaceMesh and Hands for active landmark-based challenge checks

4.5 VLM Extension Architecture

The newest system architecture adds a VLM hybrid layer as an additive branch that does not alter the original AuthPipeline decision sequence. The VLM layer runs only after the traditional video path grants access, making it a conservative second opinion layer rather than a replacement for the numeric pipeline.

File	Role
backend/app/vlm_routes.py	FastAPI router mounted at /api/v1/vlm
backend/app/vlm_pipeline.py	Composes AuthPipeline with VLM reasoning layer
backend/app/models/vlm_reasoner.py	Loads Qwen or moondream, parses structured JSON judgments
backend/app/vlm_config.py	Model IDs, thresholds, fusion weights, prompt templates
backend/app/db/vlm_models.py	vlm_registrations SQLAlchemy table definition
frontend/src/pages/VLMRegister.jsx	5-second VLM registration browser flow
frontend/src/pages/VLMLogin.jsx	5-second VLM login and natural-language reasoning display

5. Methodology

5.1 Registration Methodology

User registration builds a stable biometric template from multiple face frames. The multi-frame averaging strategy improves template quality compared to single-image enrollment by reducing noise from expression changes, lighting variation, and slight pose differences. The registration pipeline proceeds through the following steps:

9. User provides username and email, then starts camera capture of five JPEG frames.
10. Each frame is decoded from JPEG to OpenCV image array and validated for minimum resolution.
11. YuNet face detection is applied with confidence, size, yaw, and pitch validation.
12. Five-landmark geometric alignment produces a canonical 112x112 face crop.
13. A quick passive liveness score filters out obviously spoofed frames before embedding extraction.
14. ArcFace embedding extraction is performed via ONNX Runtime, producing a 512-dimensional vector.
15. Accepted embeddings from all valid frames are averaged and L2-normalized to form the template.
16. The normalized template is encrypted with AES-256-GCM and stored in SQLite with user metadata.

5.2 Authentication Methodology

Video-based authentication uses a 4-second webcam recording rather than a still image. This enables temporal liveness checks including optical flow, micro-movement detection, and rPPG pulse estimation. The middle frame of the video sequence is selected for identity matching because it typically avoids first-frame camera adjustment artifacts and final-frame movement blur.

17. User enters username; backend loads user record and decrypts stored ArcFace embedding.
18. User records a 4-second webcam video; the video is submitted to the backend via multipart form.
19. Backend decodes the video into frames and selects the middle frame for identity matching.
20. YuNet detection and alignment are applied to the middle frame, followed by ArcFace embedding extraction.
21. Cosine similarity is computed between the new embedding and the decrypted stored template.
22. Liveness score is fused over the aligned face crop and full frame sequence temporal signals.
23. Deepfake probability is fused over the aligned face and full frame sequence artifact signals.
24. The decision engine applies all gates sequentially; a JWT is issued only if all gates pass.

5.3 Decision Engine Methodology

The decision engine applies a sequence of independent gates. All gates must pass for authentication to succeed. Any failed gate produces a specific denial reason and relevant threat flag, making authentication decisions transparent and auditable.

Gate	Check	Threshold	Denial Reason
1	Camera source validation	Virtual camera rejected	virtual_camera
2	Face detection quality	Confidence, size, pose	no_face
3	Liveness score	>= 0.70	liveness_fail
4	Deepfake probability	<= 0.30	synthetic_face
5	Instruction challenge (if required)	Required actions verified	instruction_fail
6	ArcFace cosine similarity	>= 0.40	identity_mismatch
GRANT	All gates passed	JWT issued	-

5.4 VLM Hybrid Flow

The VLM hybrid flow extends traditional authentication with a semantic reasoning step that runs only after a traditional grant. This conservative design ensures that traditional numeric gates serve as the first line of defense, and VLM compute is not spent on attempts already denied by liveness, deepfake, or similarity checks.

- Traditional video authentication pipeline runs first through all six gates.
- If DENY: traditional denial reason is returned and VLM is skipped entirely.
- If GRANT: registration reference frames are loaded and authentication frames are extracted.
- VLM comparison is run: same_person, is_live, and is_authentic are evaluated.
- Final score fused as: final = 0.60 * traditional_confidence + 0.40 * vlm_overall_confidence.
- VLM veto is applied if vlm_confidence >= 0.85 and a concern is flagged.
- Final decision returned along with natural-language reasoning from the VLM.

6. AI Models and Techniques

6.1 YuNet Face Detector

YuNet is integrated with OpenCV through cv2.FaceDetectorYN. It is a lightweight real-time face detector that provides a bounding box, five facial landmarks, and a confidence score per detected face. The five landmarks (two eye corners, nose tip, and two mouth corners) are used to perform a similarity transform alignment of the face to a canonical 112x112 crop required by ArcFace.

YuNet was selected because it is CPU-friendly, avoids heavy detector dependencies, and provides the landmark output needed for ArcFace alignment without an additional landmark detection step. Detections are validated against thresholds for confidence, face size relative to image, yaw, and pitch before acceptance.

6.2 ArcFace Face Recognition

ArcFace generates normalized 512-dimensional embeddings that represent facial identity. The runtime model is w600k_r50.onnx loaded with ONNX Runtime. ArcFace uses additive angular margin loss during training to maximize inter-class separability and minimize intra-class variance in the embedding space. Identity comparison uses cosine similarity, which measures the angle between embedding vectors and is scale-invariant.

During registration, embeddings from multiple aligned face frames are averaged and L2-normalized to produce a stable template. During authentication, the extracted embedding is compared with the decrypted stored template. ONNX Runtime was used instead of framework-specific inference to simplify deployment and reduce runtime dependencies.

6.3 Liveness Detection

The liveness detection layer combines multiple signals to estimate whether a captured face appears live rather than a spoof. The weighted fusion aggregates both single-frame and multi-frame temporal cues:

- MobileNetV3-Small CNN features: Lightweight feature extractor trained on ImageNet, used for texture and high-frequency pattern analysis. Can be fine-tuned with the included training script.
- Texture analysis: Local Binary Pattern-inspired texture measures to detect printed photo artifacts and screen-display patterns.
- Color distribution analysis: Examines RGB distribution patterns characteristic of printed or screen-displayed faces.
- Moire pattern detection: Frequency-domain analysis to detect interference patterns from screen displays.
- Optical flow: Motion between consecutive video frames to verify natural head movement and reject static replays.
- Micro-movement detection: FaceMesh landmark displacement to verify subtle liveness-consistent motion patterns.
- rPPG pulse estimation: Green channel variation in forehead regions to detect physiological pulse signals absent from printed or synthetic faces.
- Optional instruction score: Challenge-response verification result incorporated into the liveness fusion score.

6.4 Deepfake Detection

The deepfake detection layer uses a hybrid approach combining frequency-domain analysis, CNN feature analysis, and handcrafted artifact checks to estimate synthetic face risk:

- FFT spectral analysis: Frequency domain anomalies characteristic of GAN-generated faces, including periodic patterns and abnormal frequency distributions.
- EfficientNet-B0 feature analysis: CNN feature statistics from the ImageNet-pretrained model analyzed for deepfake-associated distributional signatures.
- Boundary artifact detection: Blending edge artifacts common in face swap outputs and inpainting-based deepfakes.
- Eye reflection consistency: Real faces produce consistent corneal reflections; synthetic faces often show inconsistencies or missing reflections.
- Skin uniformity analysis: Smoothing artifacts characteristic of generative models applied to skin regions.
- RGB channel correlation: Correlation patterns that differ between real and synthetically generated images.
- Temporal flicker: Frame-to-frame inconsistency in video sequences that indicates synthetic generation artifacts.

6.5 MediaPipe Challenge Verification

MediaPipe FaceMesh and Hands are used for optional active challenge verification. The challenge system issues two random instructions from a predefined catalog and verifies user compliance through landmark-derived rules. The challenge is issued after the session begins, making replay attacks substantially harder since attackers cannot know the required actions in advance.

Supported verifications include: blink detection, gaze direction changes, head pose changes, mouth movements, specific hand gestures, and hand-to-face proximity checks. Challenge instructions degrade gracefully when MediaPipe is unavailable.

6.6 VLM Reasoning Layer

The Vision Language Model layer adds qualitative semantic reasoning to the numeric biometric pipeline. Two models are supported depending on hardware availability:

Model	When Used	Capability
Qwen2.5-VL-3B-Instruct	CUDA with sufficient VRAM	Higher-capability multi-image visual reasoning
moondream2	CPU or memory-constrained execution	Lightweight visual reasoning fallback

The VLM receives stored registration reference frames alongside current authentication frames and returns a structured JSON judgment covering identity comparison, liveness assessment, authenticity

evaluation, an overall confidence score, natural-language reasoning, and any red flags identified. The parser accepts direct JSON, JSON within markdown code fences, or partial JSON-like text with regex fallback for robustness.

7. Technology Stack

The complete technology stack used across all layers of the system is summarized below:

Category	Technologies
Frontend	React 18, Vite, Axios, React Router
Backend	FastAPI, Uvicorn, Python multipart handling
Computer Vision	OpenCV, NumPy, SciPy
AI Inference Runtime	ONNX Runtime, PyTorch, Torchvision
Face Detection	OpenCV YuNet ONNX (cv2.FaceDetectorYN)
Face Recognition	ArcFace w600k_r50.onnx via ONNX Runtime
Liveness Detection	MobileNetV3-Small, texture/color checks, rPPG, optical flow
Deepfake Detection	FFT, EfficientNet-B0 features, handcrafted artifact checks
Challenge Verification	MediaPipe FaceMesh and Hands
VLM Reasoning	Qwen2.5-VL-3B-Instruct / moondream2 (HuggingFace Transformers)
Database	SQLite, SQLAlchemy ORM
Security	AES-256-GCM, RS256 JWT, SlowAPI rate limiting, CORS policy

8. Complete System Flow

8.1 Traditional Registration Flow

The traditional registration flow captures five still frames through the browser camera interface. Each frame undergoes face detection, validation, alignment, and ArcFace embedding extraction independently. Only frames passing quality validation contribute to the final averaged template, making registration robust to occasional poor-quality frames.

The averaged and normalized embedding is encrypted with AES-256-GCM before database storage. The encrypted ciphertext, authentication tag, and initialization vector are stored together, ensuring that the database does not contain usable plaintext biometric data. The registration endpoint returns the user ID, liveness score, face quality metric, and status.

8.2 Traditional Authentication Flow

Authentication begins with the user recording a 4-second video. The backend decodes the video into frames, selects the middle frame for identity matching, and processes the full frame sequence for liveness and deepfake analysis. This design separates the concerns of identity matching and anti-spoofing analysis: the middle frame provides a stable single-frame representation for cosine similarity computation, while the full sequence enables temporal signal extraction.

The decision engine then applies all required gates sequentially. Each gate can independently deny access with a specific reason, ensuring that a spoofed input cannot bypass the system by passing only the identity check. On successful authentication, a signed RS256 JWT is issued for the user session.

8.3 Challenge Flow

The optional challenge flow adds an active verification layer. The client first requests a challenge from GET /api/v1/challenge, which returns two random instructions, a challenge ID, and a time-to-live value. The challenge is stored server-side with its TTL.

The client records separate videos for each instruction and submits them with the challenge ID. The backend validates the challenge TTL, decodes both videos, verifies each instruction against MediaPipe-derived landmarks, and incorporates the instruction score into the overall authentication decision. This makes replay attacks substantially harder since instructions are randomized per session.

8.4 VLM Hybrid Flow

The VLM hybrid flow extends the traditional authentication with a semantic reasoning step that runs only after a traditional grant. Traditional numeric gates serve as the first line of defense. If the traditional pipeline denies access, the VLM is not invoked. If the traditional pipeline grants access, the VLM provides a second opinion and natural-language explanation.

The final fusion score is computed as: final = 0.60 * traditional_confidence + 0.40 * vlm_overall_confidence. A VLM veto mechanism can override a traditional grant if the VLM confidence is high (>= 0.85) and a concern is flagged. This conservative design ensures the original pipeline remains the primary security layer.

9. Security Features

9.1 Biometric Template Encryption

ArcFace face embeddings are encrypted with AES-256-GCM before storage. AES-256-GCM provides both confidentiality and integrity authentication: the encryption key is 256 bits, and the GCM mode produces an authentication tag that detects any tampering with the ciphertext. A unique initialization vector is generated per encryption operation, preventing identical plaintexts from producing identical ciphertexts.

The encryption key is provided through the FACE_AUTH_AES_KEY environment variable as a 64-character hexadecimal string. Face embeddings are among the most sensitive category of biometric data because they cannot be changed if compromised. The system treats encrypted embeddings as the primary biometric protection measure.

9.2 JWT Authentication Tokens

Successful authentication returns an RS256 JWT. RS256 uses asymmetric cryptography with a private key for signing and a public key for verification. This means the backend can distribute the public key to downstream services for token verification without exposing the signing key. The JWT contains a subject claim identifying the authenticated user and an expiry claim.

9.3 Rate Limiting and CORS

SlowAPI rate limiting restricts sensitive endpoints to 5 requests per minute per source IP. Requests exceeding this limit receive HTTP 429 responses, mitigating brute-force and enumeration attacks.

CORS policy is configured to permit only local development origins in the current implementation. Production deployment requires stricter CORS configuration aligned with the actual deployment domain.

9.4 Anti-Injection Guard

The anti-injection guard is designed to detect virtual or software-based camera sources by inspecting camera metadata, PRNU-like noise variance, and frame characteristics. This layer addresses the attack vector where an attacker uses a virtual camera driver to inject fabricated or pre-recorded video frames, bypassing physical presence requirements.

9.5 Audit Logging

Every authentication attempt is recorded in the SQLite auth_logs table regardless of outcome. Each record stores the attempt timestamp, source IP, authentication decision, denial reason if applicable, all model scores, and all threat flags. This audit trail supports post-incident analysis and provides evidence for security reviews.

Security Control	Implementation	Protects Against
AES-256-GCM encryption	Template encrypted before storage	Database breach exposing raw biometrics
RS256 JWT	Signed token after successful auth	Session hijacking, token forgery
Rate limiting	5 requests/minute via SlowAPI	Brute force and credential stuffing
CORS policy	Local origins only in development	Cross-origin request forgery
Audit logging	All attempts in auth_logs	Incident investigation, accountability
Multi-layer pipeline	6 independent gates	Single-model bypass attacks

10. Database Design

10.1 Schema Overview

The system uses SQLite through SQLAlchemy for all persistent storage. The database is located at backend/data/auth.db. Four tables are used: three in the traditional pipeline and one additional table for VLM storage.

10.2 Users Table

Field	Purpose
id	Primary key, auto-incremented user identifier
username	Unique login identifier for authentication lookup
email	Unique email address for account identification
embedding_enc	AES-256-GCM encrypted 512-dimensional face template
face_quality	Average registration detection confidence score
created_at	Registration timestamp

10.3 Auth Logs Table

Field	Purpose
user_id	User associated with the authentication attempt
timestamp	Attempt time for audit trail
liveness_score	Final liveness fusion score for the attempt
deepfake_score	Final deepfake probability estimate
similarity_score	ArcFace cosine similarity value against stored template
threat_flags	JSON array of detected threat flags (e.g., virtual_camera, synthetic_face)
decision	GRANT or DENY outcome
denial_reason	Human-readable reason for denial if applicable

10.4 VLM Reference Frame Storage

VLM registration selects the best-quality frames from the registration video and stores them as JPEG files under backend/data/vlm_ref_frames/{user_id}/. The vlm_registrations table stores the file paths and frame count for each registered user. For production deployment, VLM reference frames should

be encrypted at rest or stored in a protected object store with access logging, as they contain biometric facial imagery.

11. Frontend and User Experience

11.1 Traditional Registration

The registration page (Register.jsx) presents a form for username and email entry. After the form is completed, the browser requests camera permission. The CameraCapture component displays a live camera preview and captures five still JPEG frames automatically or on user prompt. Captured frames are assembled into a multipart form request and submitted to the registration endpoint.

The page displays registration success with the liveness score, face quality metric, and assigned user ID. Error handling covers camera permission denial, face detection failures, and server errors.

11.2 Traditional Video Login

The login page (Login.jsx) accepts a username and opens the camera through the VideoRecorder component. The component records a 4-second WebM video with a visible countdown timer. After recording completes, the video is submitted automatically. While the backend processes the request, the page shows a loading indicator.

On response, the page renders animated score bars for Face Match, Liveness, and Deepfake probability, along with the authentication decision, confidence value, any threat flags, and processing time. Denial reasons are displayed clearly with actionable guidance for the user.

11.3 VLM Registration and Login

The VLM registration page (VLMRegister.jsx) runs a 3-second preparation countdown followed by a 5-second video recording. The slightly longer recording gives the pipeline more frames for reference selection and quality evaluation.

VLM login (VLMLogin.jsx) follows the same recording flow and displays both traditional pipeline scores and VLM scores including VLM Identity, VLM Liveness, and VLM Authenticity. The page also shows the VLM model name, whether VLM override was applied, whether registration reference frames were available, and the natural-language reasoning returned by the VLM model.

11.4 Score Interpretation Guide

Score	Good Direction	Meaning
Face Match	Higher is better	ArcFace cosine similarity to stored template
Liveness	Higher is better	Evidence that the face appears live and not a spoof
Deepfake	Lower is better	Estimated synthetic or manipulation risk probability
VLM Identity	Higher is better	VLM confidence that reference and auth frames show the same person
VLM Liveness	Higher is better	VLM confidence that auth frames show a live person
VLM Authenticity	Higher is better	VLM confidence that auth frames are not spoofed or manipulated

12. API Reference

12.1 Core Endpoints

Method	Endpoint	Purpose	Auth
GET	/health	Server and pipeline status check	No
POST	/api/v1/register	Register user from face images	No
POST	/api/v1/authenticate	Authenticate from single image (legacy)	No
POST	/api/v1/authenticate/video	Authenticate from webcam video (primary)	No
GET	/api/v1/challenge	Issue random active challenge	No
POST	/api/v1/authenticate/challenge	Authenticate with challenge videos	No
GET	/api/v1/instructions	List instruction catalog	No
GET	/api/v1/users/{id}/history	Last 10 attempts for user	No

12.2 VLM Endpoints

Method	Endpoint	Purpose
POST	/api/v1/vlm/register	VLM registration with reference frame storage
POST	/api/v1/vlm/authenticate	Traditional + VLM hybrid authentication
GET	/api/v1/vlm/status	VLM model readiness and hardware info

12.3 Authentication Response Structure

A successful video authentication response includes the following JSON fields:

```
{ "authenticated": true, "confidence": 0.84, "threat_flags": [], "scores": { "liveness": 0.86, "deepfake": 0.08, "similarity": 0.78, "injection": 1.0 }, "processing_time_ms": 1420.5, "jwt_token": "..." }
```

The VLM authentication response additionally includes vlm_reasoning, vlm_model_used, vlm_invoked, vlm_override, has_vlm_refs, a nested vlm scores object, and traditional_decision and traditional_confidence fields for comparison.

13. Setup and Operations

13.1 Prerequisites

- Python 3.11 or 3.12 recommended for the backend environment.
- Node.js and npm for the React frontend build and development server.
- A webcam for full browser-based demo registration and authentication flows.
- Optional: CUDA-capable GPU for faster VLM inference (Qwen2.5-VL-3B-Instruct).
- ONNX model files: w600k_r50.onnx (ArcFace) and face_detection_yunet.onnx (YuNet).

13.2 Backend Setup

```
cd backend python -m venv venv .\venv\Scripts\Activate.ps1 pip install -r requirements.txt $env:FACE_AUTH_AES_KEY = "0000...000"  # 64 hex chars $env:PYTHONPATH = "." uvicorn app.main:app --reload --port 8000
```

13.3 Frontend Setup

```
cd frontend npm install npm run dev # Open: http://localhost:5173
```

13.4 Environment Variables

Variable	Default	Purpose
FACE_AUTH_AES_KEY	64 zeros (dev only)	AES-256-GCM key as 64-char hex string
JWT_PRIVATE_KEY	backend/keys/private.pem	RS256 private key path for JWT signing
JWT_PUBLIC_KEY	backend/keys/public.pem	RS256 public key path for JWT verification
VLM_MODEL	auto	VLM model: auto, qwen, moondream, disabled
VLM_MAX_RAM_GB	5.0	Soft RAM target for automatic VLM model selection

14. Results and Evaluation

14.1 Runtime Score Outputs

The system returns detailed per-attempt results for every authentication request. These results enable real-time monitoring and post-hoc analysis of authentication decisions. Each attempt record includes the authentication decision, overall confidence, liveness score, deepfake probability, ArcFace similarity score, injection confidence, active threat flags, and processing time in milliseconds.

14.2 Evaluation Metrics Supported

The repository includes an evaluation script capable of producing the following metrics when a suitable labeled dataset is provided:

Metric	Description
AUC-ROC	Area under the receiver operating characteristic curve
FAR	False Acceptance Rate — proportion of impostors incorrectly accepted
FRR	False Rejection Rate — proportion of genuine users incorrectly rejected
EER	Equal Error Rate — operating point where FAR equals FRR
Precision / Recall / F1	Classification quality metrics for the binary authentication decision
Mean / P95 Latency	Average and 95th-percentile processing time per authentication request

14.3 Evaluation Command

```
python -m training.evaluate --data_root data/test --weights_dir weights --model all
```

14.4 Observed System Behavior

During local development testing, the system demonstrated the following behavioral properties:

- Registration successfully processes multiple frames and reports face quality and liveness scores per attempt.
- Video authentication returns differentiated scores across genuine, printed-photo, and screen-replay inputs when tested manually.
- Denial reasons are specific and actionable, identifying which gate failed and what corrective action the user should take.
- VLM reasoning returns natural-language explanations that match visual observations when reference frames are available.

- Processing time is dominated by model loading on first inference; subsequent requests process significantly faster due to model caching.

Note: Dataset-backed benchmark values (FAR, FRR, AUC) are not committed in the current repository. The runtime scores are available per authentication attempt and can be accumulated over a test run for statistical evaluation.

15. Limitations

15.1 Technical Limitations

- Thresholds for liveness, deepfake, and similarity are not calibrated against a documented benchmark dataset. The current values are reasonable development defaults but require calibration for any specific deployment context.
- Lighting conditions, camera quality, face pose, and motion blur significantly affect liveness and deepfake scores. Poor capture conditions may cause false rejections of genuine users.
- HTTP upload endpoints do not have access to live camera device handles, preventing physical camera anti-injection validation for upload-based flows.
- The runtime deepfake detection module uses EfficientNet-B0 features while the included training script supports training EfficientNet-B4 classifiers. Fine-tuned B4 artifacts require the runtime module to be updated for use.
- rPPG pulse estimation requires a sufficient number of video frames and stable lighting. Short videos or poor lighting conditions reduce the reliability of this signal.
- MediaPipe FaceMesh and Hands availability affects challenge verification. Challenge instructions degrade gracefully when MediaPipe is unavailable but provide no active security in that case.
- Some MediaPipe challenge checks are heuristic and may not be robust across all lighting and positioning conditions.

15.2 VLM-Specific Limitations

- VLM inference can be slow, particularly on CPU. The first VLM call may take significantly longer due to model weight loading from disk.
- VLM output is probabilistic and may produce inconsistent judgments for visually ambiguous inputs.
- VLM reference frames are stored as unencrypted JPEG files in the current implementation. Production deployments should encrypt these files at rest.
- VLM reasoning is returned in the API response but is not currently stored in a dedicated VLM audit table for long-term history.
- Hardware dependency: Qwen2.5-VL-3B-Instruct requires a CUDA-capable GPU with sufficient VRAM for reasonable inference latency. Moondream2 provides a CPU fallback with reduced capability.

15.3 Engineering and Deployment Limitations

- Production deployment requires HTTPS, stronger secret management, database migration tooling, monitoring infrastructure, and production-grade CORS configuration.
- The Docker setup requires retesting against the current dependency set before relying on containerized deployment.
- No automated tests for API endpoints or decision engine logic are currently committed to the repository.
- The default AES key is suitable only for development and must be replaced with a securely generated key for any real deployment.
- No committed benchmark dataset means that specific numeric accuracy claims cannot currently be independently verified.

17. Conclusion

This project successfully demonstrates a comprehensive AI-based facial authentication system that addresses the core vulnerability of face recognition alone: the inability to distinguish a live, genuine face from a spoofed, replayed, or synthetically generated one. The layered architecture combines face detection, identity matching, liveness detection, deepfake risk estimation, optional active challenge verification, and optional Vision Language Model reasoning into a coherent, explainable pipeline.

The key contributions of the project include the multi-gate decision engine that requires agreement from independent AI signals, the use of video-based authentication for temporal liveness analysis, the integration of AES-256-GCM encrypted biometric template storage, the RS256 JWT token-based session management, the comprehensive audit logging of all authentication attempts, and the VLM hybrid reasoning layer that adds natural-language explainability on top of the numeric biometric pipeline.

The multi-layer design means that no single model failure can result in a false acceptance. An attacker must simultaneously fool the face similarity check, the liveness detector, the deepfake detector, and optionally the active challenge system. This dramatically raises the bar for successful spoofing attacks compared to single-signal systems.

The system combines several academic disciplines into a working implementation: digital signal processing through FFT and rPPG analysis, computer vision through face detection and alignment, deep learning through ArcFace embeddings and CNN feature analysis, cybersecurity through spoofing defense, encryption, JWT issuance, and audit logging, and full-stack software engineering through the React and FastAPI implementation.

The VLM extension demonstrates how large multimodal models can serve a supporting role in biometric security by providing interpretable reasoning about visual evidence that complements the deterministic numeric pipeline. With additional dataset-backed benchmarking, fine-tuned model integration, and production security hardening, the architecture presented here provides a solid foundation for deployment in real-world biometric authentication scenarios.

18. Appendix A: Model Summary Table

Component	Model / Method	Runtime	Purpose
Face Detection	YuNet ONNX	OpenCV FaceDetectorYN	Bounding box, landmarks, confidence
Face Recognition	ArcFace w600k_r50.onnx	ONNX Runtime	512-dim identity embeddings
Liveness (CNN)	MobileNetV3-Small	PyTorch / Torchvision	Texture and feature-based liveness
Liveness (Signal)	rPPG green channel	NumPy	Pulse estimation from video
Liveness (Motion)	Optical flow	OpenCV	Movement and micro-motion analysis
Challenge Verify	MediaPipe FaceMesh + Hands	MediaPipe	Landmark-based instruction check
Deepfake (Freq)	FFT spectral analysis	NumPy / SciPy	Frequency artifact detection
Deepfake (CNN)	EfficientNet-B0 features	PyTorch	CNN feature anomaly detection
VLM (Primary)	Qwen2.5-VL-3B-Instruct	Transformers + CUDA	Multi-image visual reasoning
VLM (Fallback)	moondream2	Transformers + CPU	Lightweight visual reasoning fallback

19. Appendix B: Denial Reasons Reference

Denial Reason	Triggering Condition	Suggested Action
virtual_camera	Camera source appears non-physical (virtual driver detected)	Use a physical webcam device
no_face	No face detected at acceptable quality / confidence	Improve lighting and face positioning
liveness_fail	Liveness fusion score below 0.70 threshold	Use live video; avoid spoofing materials
synthetic_face	Deepfake probability above 0.30 threshold	Submit a live, unmanipulated face
instruction_fail	Active challenge instructions not verified by MediaPipe	Follow the displayed instructions clearly
identity_mismatch	ArcFace cosine similarity below 0.40 threshold	Ensure correct user is registered
no_frames	Video could not be decoded into usable frames	Re-record with a compatible browser

20. References

[1] Deng, J., Guo, J., Xue, N., & Zafeiriou, S. (2019). ArcFace: Additive Angular Margin Loss for Deep Face Recognition. IEEE/CVF CVPR.

[2] Wu, W., et al. (2023). YuNet: A Tiny Millisecond-level Face Detector. Machine Intelligence Research.

[3] Wu, W., Qian, C., Yang, S., et al. (2021). RetinaFace: Single-Shot Multi-Level Face Localisation in the Wild. IEEE/CVF CVPR.

[4] Howard, A., et al. (2019). Searching for MobileNetV3. IEEE/CVF International Conference on Computer Vision (ICCV).

[5] Tan, M., & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. ICML.

[6] Li, X., et al. (2020). Face Anti-Spoofing: Model Matters, So Does Data. IEEE/CVF CVPR.

[7] Lugas, J., et al. (2006). Digital Camera Identification from Sensor Pattern Noise. IEEE Transactions on Information Forensics and Security.

[8] Lugaresi, C., et al. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv:1906.08172.

[9] Bao, H., et al. (2021). FaceX-Zoo: A PyTorch Toolbox for Face Recognition. ACM International Conference on Multimedia.

[10] Rossler, A., et al. (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. IEEE/CVF ICCV.

[11] De Haan, G., & Jeanne, V. (2013). Robust Pulse Rate from Chrominance-Based rPPG. IEEE Transactions on Biomedical Engineering.

[12] Qwen Team. (2024). Qwen2.5-VL Technical Report. Alibaba Cloud. arXiv preprint.

[13] NIST FRVT (Face Recognition Vendor Test). National Institute of Standards and Technology. https://www.nist.gov/programs-projects/face-recognition-vendor-testing-frvt